

Analysis and Problem Identification

1.1 Understanding of the Case

A retail system frequently sorts inventory based on price and customer demand. As the number of products increases, the sorting method must provide good speed, controlled memory usage, and reliable performance.

1.2 Core Problem

The system must efficiently sort small to very large inventory datasets without excessive execution time or memory consumption.

1.3 Objectives

1. Compare Merge Sort and Quick Sort for retail inventory.
2. Apply the Divide and Conquer technique.
3. Analyze time complexity, memory usage, and scalability.
4. Design and justify the most suitable sorting approach.

1.4 Assumptions and Constraints

- Each product has Product ID, Price and Demand.
- The dataset may range from thousands to millions of records.
- Sorting may be performed frequently.
- Equal prices or demand values may occur.
- The solution should avoid poor performance on large or sorted data.

1.5 Sample Input / Output

Product	Price	Demand
Keyboard	₹1200	450
Mouse	₹700	620
Monitor	₹8500	280
Headset	₹1800	510
Webcam	₹2500	390

Price ascending: Mouse (₹700), Keyboard (₹1200), Headset (₹1800), Webcam (₹2500), Monitor (₹8500).

2. Proposed Solution / Methodology

2.1 Merge Sort

Merge Sort divides the inventory into two halves, recursively sorts both halves, and merges them. Its time complexity is $O(n \log n)$ in the best, average and worst cases. It requires $O(n)$ extra memory and is stable.

2.2 Quick Sort

Quick Sort selects a pivot, partitions the records into smaller and larger elements, and recursively sorts the partitions. Its average time complexity is $O(n \log n)$, but its worst case is $O(n^2)$. An in-place implementation uses $O(\log n)$ average recursion space and is generally not stable.

2.3 Divide and Conquer Comparison

Feature	Merge Sort	Quick Sort
Divide	Two halves	Pivot partition
Best / Average	$O(n \log n)$	$O(n \log n)$

Worst	$O(n \log n)$	$O(n^2)$
Extra Space	$O(n)$	$O(\log n)$ average
Stable	Yes	Generally No
Practical Strength	Predictable	Fast in-memory

2.4 Procedure

5. Load inventory data.
6. Select price or demand as the sorting key.
7. Check data size and requirements.
8. Select Merge Sort or Quick Sort.
9. Sort using Divide and Conquer.
10. Verify and display the sorted inventory.
11. Measure execution time for comparison.

2.5 Mathematical Formulation

Merge Sort: $T(n) = 2T(n/2) + O(n) = O(n \log n)$.

Quick Sort: balanced partition gives $O(n \log n)$; highly unbalanced partition gives $O(n^2)$. Randomized pivot selection reduces the risk of the worst case.

3. Implementation / Execution

3.1 Tools

- Python
- Visual Studio Code / PyCharm
- CSV file or database
- Matplotlib for performance graphs

3.2 Sample Implementation

```
def merge_sort(a):
    if len(a) <= 1:
        return a
    m = len(a)//2
    left = merge_sort(a[:m])
    right = merge_sort(a[m:])
    return merge(left, right)

def quick_sort(a):
    if len(a) <= 1:
        return a
    pivot = a[len(a)//2]
    left = [x for x in a if x < pivot]
```

```

mid = [x for x in a if x == pivot]

right = [x for x in a if x > pivot]

return quick_sort(left) + mid + quick_sort(right)

```

3.3 Performance Testing

Dataset Size	Merge Sort	Quick Sort
1,000	Measure execution time	Measure execution time
10,000	Measure execution time	Measure execution time
50,000	Measure execution time	Measure execution time
100,000	Measure execution time	Measure execution time
500,000	Measure execution time	Measure execution time

Actual benchmark values should be obtained by running the program on the computer rather than using assumed values.

4. Results and Analysis

4.1 Memory and Speed

Quick Sort generally uses less auxiliary memory and can be very fast for in-memory inventory. Merge Sort uses more memory but provides predictable $O(n \log n)$ performance.

4.2 Data Size

- Small datasets: both algorithms are suitable.
- Medium in-memory datasets: Quick Sort can provide excellent practical speed.
- Very large or external datasets: Merge Sort is more suitable because of its predictable performance and support for external sorting.

4.3 Stability

Merge Sort is stable, so records with equal prices or demand can retain their original order. Standard Quick Sort is generally not stable.

5. Discussion

5.1 Advantages

Merge Sort	Quick Sort
Guaranteed $O(n \log n)$	Fast average performance
Stable	Lower auxiliary memory
Reliable for large data	Good for in-memory data
Suitable for external sorting	Efficient with good pivot selection

5.2 Limitations

- Merge Sort: requires $O(n)$ additional memory and involves merging overhead.
- Quick Sort: can become $O(n^2)$ with poor pivot selection and is generally unstable.

5.3 Proposed Solution

Use an adaptive approach. For normal in-memory inventory, use Quick Sort with randomized pivot selection. Use Merge Sort when stable ordering, guaranteed performance, or very large/external data is required.

6. Conclusion

The case study compared Merge Sort and Quick Sort for retail inventory sorting using Divide and Conquer principles. Both provide $O(n \log n)$ average performance, but they differ in memory usage, stability, and worst-case behavior.

Quick Sort is recommended for normal in-memory inventory because of its strong practical speed and lower auxiliary memory. Merge Sort is recommended for stable, predictable, or very large/external sorting.

Therefore, an adaptive solution that selects the algorithm according to data size, memory availability, and stability requirements provides the best balance of speed, scalability, and reliability.

Future Scope

- Automatic algorithm selection based on dataset size.
- Randomized or median-based pivot selection.
- Three-way partitioning for duplicate values.
- External Merge Sort for extremely large inventory.
- Performance dashboard and parallel sorting.

Student Signature: _____

Faculty Signature: _____